



STRATA

Browse in context.

DOC 01 · SOFTWARE DEVELOPMENT LIFE CYCLE

Software Development Life Cycle

How Strata gets built: methodology, roles, environments, testing, delivery, and risk management for a native browser project.

Version: 1.0

Date: September 2026

Status: Draft for team review

Owner: Engineering

strata — local-first browser
confidential — internal working document

CONTENTS

01 Purpose & Methodology

02 Roles & Responsibilities

03 Environment & Branching Strategy

04 Testing Strategy

05 CI/CD & Release Process

06 Risk Management

07 Change Management & Documentation

Purpose & Methodology

This document defines the Software Development Life Cycle (SDLC) for **Strata**, a privacy-first, Chromium-based desktop browser built around **Continuum** — a local system for remembering browsing context. Strata's engineering surface spans four layers (React UI, Rust/Tauri application core, a C/C++ CEF bridge, and Chromium itself), so the SDLC is written for a team building a genuine browser, not a web app — native process management, multi-process security boundaries, and platform packaging all carry real schedule risk.

CHOSEN MODEL

A **phase-gated foundation with agile execution inside each phase**. The roadmap (Phase 0 → Phase 7, detailed in the Implementation Plan) behaves like a series of waterfall gates because later phases are load-bearing on earlier ones (you cannot build Moments before Continuum records events; you cannot record events before CEF renders a page reliably). Inside each phase, work is run in 1–2 week iterations with standard agile ceremonies.

Why not pure Scrum, and why not pure Waterfall

Pure Scrum with a single flat backlog invites the team to build UI (React tabs, Continuum panels) before the native rendering surface is proven — a classic browser-project failure mode, since CEF + native window embedding is the single largest source of unknowns in this project. Pure Waterfall, on the other hand, defers all user-facing feedback to the very end, which is risky for a product whose core value (Continuum, Moments, Rewind) is genuinely novel and needs early validation. The hybrid model gates on *technical risk*, not on documentation sign-off.

Gate criteria

Each phase ends with a working, demoable build — not a spec review. See §7 Definition of Done.

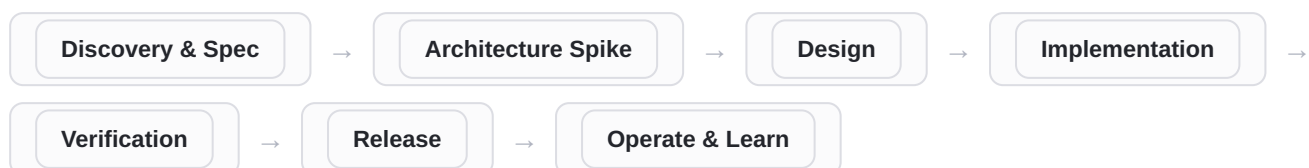
Iteration length

1–2 week sprints inside a phase; phase length itself is variable and risk-driven, not fixed.

Feedback loop

Internal dogfooding starts the moment Phase 2 (browser fundamentals) is usable daily.

SDLC stages



This cycle repeats per phase of the roadmap, and in miniature within every sprint. The "Architecture Spike" stage is unique to Strata's early phases and is deliberately called out separately from Implementation: before any React UI is built, a standalone proof of concept must show CEF creating a browser, loading a URL, and rendering it inside a native window it does not own outright (§45 of the product brief). No planning downstream of that spike is considered reliable until it passes.

Roles & Responsibilities

A small, cross-disciplinary team is assumed. Roles are written as responsibilities, not headcount — one person may hold more than one role in an early-stage team, but the native-bridge and product/design responsibilities should not be held by the same person if avoidable, since they require different judgment (systems risk vs. taste).

ROLE	OWNS	PRIMARY ARTIFACTS
Product / Design lead	PRD, UX brief, roadmap prioritization, demo scenario acceptance	PRD, UI/UX Brief, App Flow
Native / Systems engineer (Rust)	Tauri shell, BrowserManager, TabManager, WindowManager, IPC boundary	src-tauri/src/browser/*, commands/*
CEF / C++ integration engineer	native/cef_bridge, multi-process lifecycle, CefBrowser/CefFrame/CefBrowserHost usage	native/cef_bridge/*, CMakeLists.txt
Frontend engineer (React/TS)	Browser chrome: tabs, address bar, Continuum UI, Moments, command palette	frontend/src/components/*, stores/*
Continuum / data engineer	EventRecorder, StateCollector, SQLite schema & migrations, restore logic	src-tauri/src/continuum/*, storage/*
QA / Release engineer	Test plans, CI pipeline, code signing, release channels	scripts/*, CI config

STAFFING NOTE

If this is a solo or two-person build (explicitly anticipated in §41 of the product brief), treat the roles above as *hats*, and use the phase gates even more strictly — a single person context-switching between CEF C++ and React state management benefits most from not doing both in the same week.

Environment & Branching Strategy

Environments

LOCAL DEV

Debug CEF build, verbose native logging, hot-reloaded React chrome

NIGHTLY

Automated build from `main`, unsigned, internal dogfood only

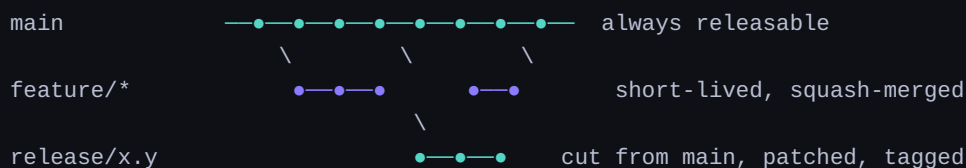
BETA

Signed builds from a release branch, opt-in external testers

STABLE

Signed, notarized/verified release artifact per OS

Branching model



Native code (Rust, C++/CEF bridge) and frontend (React) live in one monorepo per the recommended project structure. Because a broken CEF bridge blocks everyone, `main` is protected: CI must pass a native build on at least the primary target platform (Windows first, per the phased platform strategy) before merge.

Code review & quality gates

- Every PR requires one review; PRs touching `native/cef_bridge`, `commands/`, or the IPC surface require review from the native/systems owner specifically, because of the Critical IPC Rule (§4 of the TRD) — no website JavaScript may reach the filesystem through Tauri IPC.
- Frontend PRs run lint, type-check, and component tests; native PRs run `cargo test`, `cargo clippy`, and a CEF smoke build.
- Any change to the SQLite schema requires a migration file and a rollback note (see Backend Schema doc).
- UI changes touching Continuum, Moments, or the address bar require a screenshot or short recording in the PR — these are the product's differentiators and deserve visual review, not just code review.

Testing Strategy

Strata's testing surface is unusually layered for a "web app": it includes ordinary UI/unit testing, but also native-process stability, IPC boundary security, and restoration *fidelity* (did the Moment actually come back the way it left?), which is a product-correctness property, not just a bug category.

LAYER	WHAT'S VERIFIED	TOOLING
Unit — Rust	BrowserManager, TabManager, ContinuumManager logic in isolation	<code>cargo test</code>
Unit — React	Stores (Zustand), component rendering, command palette filtering	Vitest / component tests
Integration	Rust ↔ CEF bridge calls; SQLite read/write round-trips; migrations	Integration harness against a real CEF subprocess
Restoration fidelity	Save Moment → close tabs → restore → scroll/zoom/ split position match within tolerance	Scripted scenarios, manual acceptance per release
Security	No uncontrolled path from page JS → Tauri IPC → filesystem; renderer sandboxing intact	Manual pen-test pass + automated IPC allowlist test
Performance	Startup time, tab-switch latency, Continuum write overhead	Instrumented benchmark runs, see TRD §6 targets
Manual / exploratory QA	Real-world browsing session across the Demo Scenario's five site types	Human QA pass before every Beta/Stable tag

NON-NEGOTIABLE GATE

No release — nightly excluded — ships if the Critical IPC Rule test fails, or if a Continuum-recorded Moment cannot be restored for the primary demo scenario. These are treated as Sev-1, blocking defects regardless of what else is ready.

CI/CD & Release Process

```

PR opened
  |
  ├── lint + typecheck (frontend)           ── cargo fmt + clippy (native)
  |
  ▼
CI build matrix — Windows (primary) — macOS (later) — Linux (later)
  |
  ▼
cargo test · vitest · integration harness
  |
  ▼
merge to main → nightly channel (auto, unsigned)
  |
  ▼
release/x.y cut → beta channel (signed) → stable channel (signed + notarized)
  
```

Per the product brief's platform strategy (§40): "Start with Windows. Get the browser working properly on one platform before attempting three." The CI matrix reflects that — Windows is the only *required* pipeline until Phase 6/7; macOS and Linux builds are added as opt-in, non-blocking jobs once the Windows build is stable, then promoted to required before a cross-platform Stable release.

Definition of Done (per phase)

- The phase's stated goal (see Implementation Plan) is demoable end-to-end, not just unit-tested in isolation.
- No open Sev-1/Sev-2 defects in the area the phase touched.
- Automated tests exist for new native commands and new Continuum event types.
- Docs updated: schema changes reflected in the Backend Schema doc; new flows reflected in the App Flow doc.
- Performance targets for the phase (TRD §6) are met on the primary platform.

Risk Management

RISK	IMPACT	MITIGATION
CEF + native window embedding proves harder than estimated	Schedule slip across every later phase	Phase 0 is a dedicated, timeboxed spike before any UI work starts (§43, §45)
IPC surface accidentally exposes filesystem/native access to page JS	Critical security vulnerability	Explicit Critical IPC Rule, mandatory review on IPC-touching PRs, automated allowlist test
Continuum recording adds visible UI jank	Undermines the product's "fast" principle	Debounced event capture, async batched SQLite writes (§26, §35 of product brief)
Feature creep toward AI (search suggestions, summarization)	Violates a core, publicly stated brand promise	"No AI" is a written product principle; PRD non-goals section is a standing review checklist
Attempting three OS platforms simultaneously	Diluted quality everywhere, nothing shippable	Windows-first sequencing enforced at the CI and roadmap level
Restoration fidelity varies wildly by website	Users lose trust in Continuum's core promise	Explicit "browser-owned vs. page-owned state" contract communicated in-product, never overpromised (TRD §5)

Change Management & Documentation

This document set — SDLC, PRD, TRD, UI/UX Brief, App Flow, Backend Schema, and Implementation Plan — is the source of truth and is versioned together. A change to product scope (PRD) that affects data recorded (Backend Schema) or user flow (App Flow) must update all affected documents in the same change, reviewed by Product and the owning engineer together.

LIVING DOCUMENTS

These are working documents, not a one-time spec. Re-review the PRD's non-goals and the TRD's performance targets at the start of every phase gate — scope drift is the most common way small browser projects stall out.